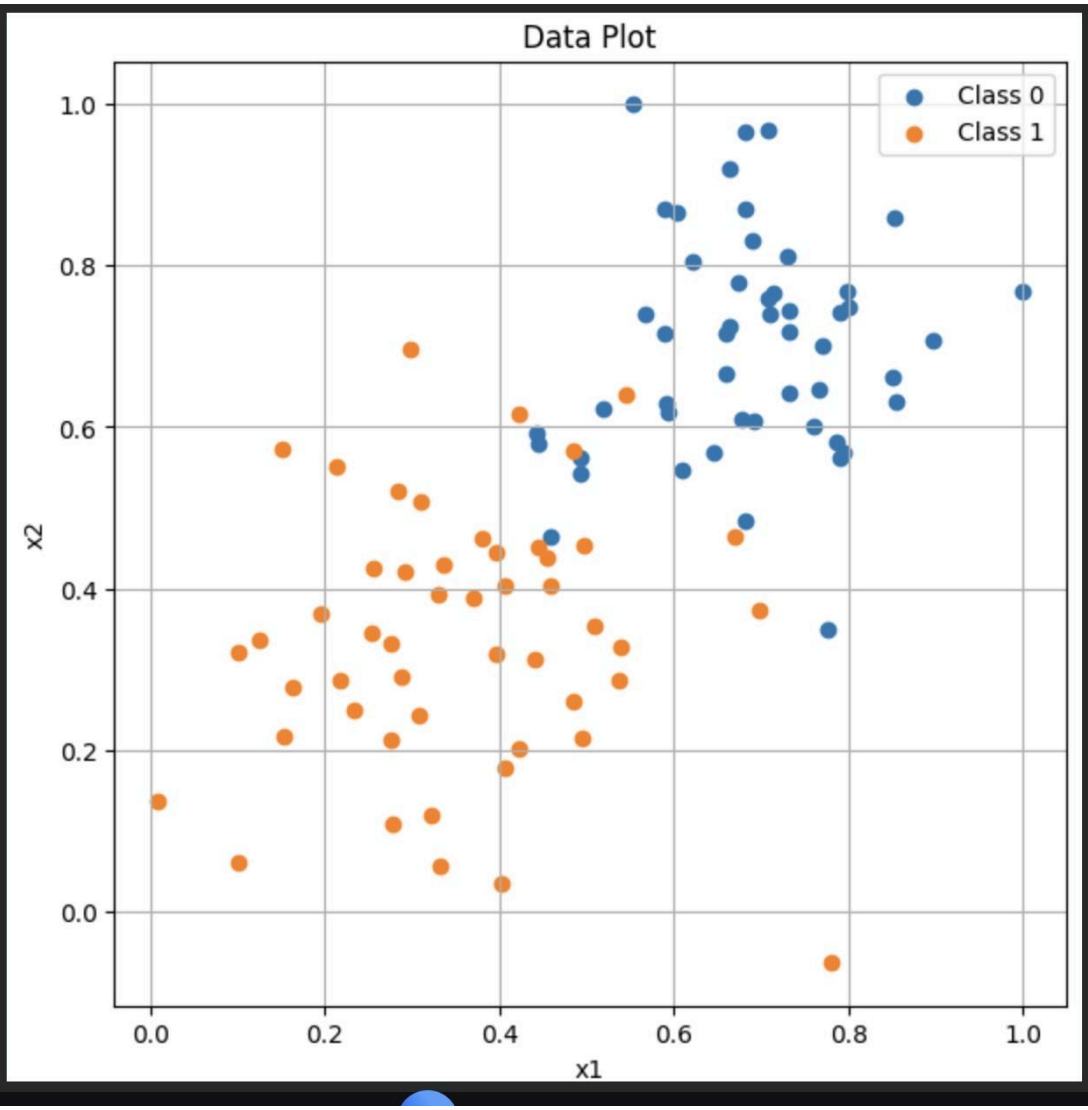
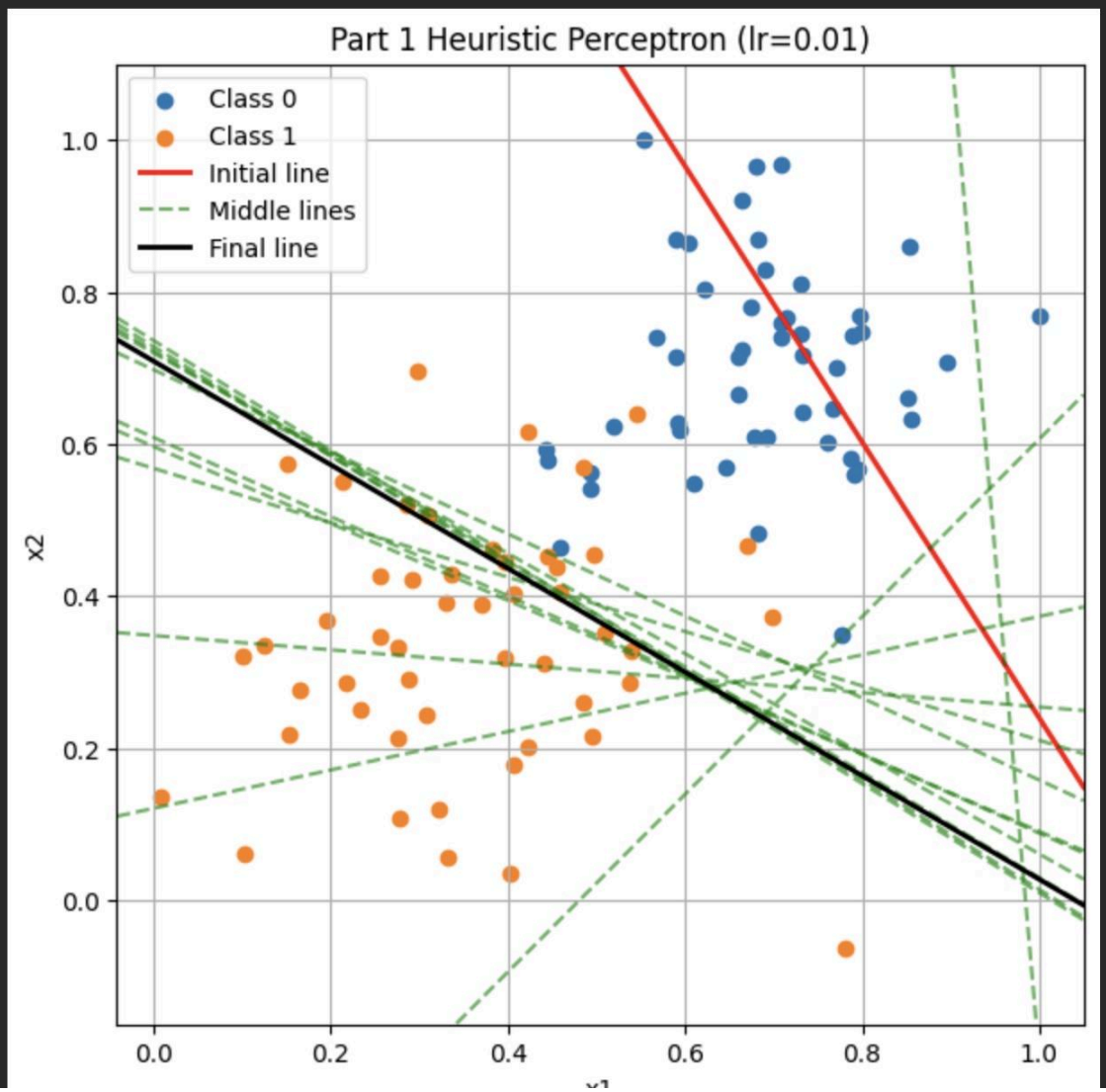
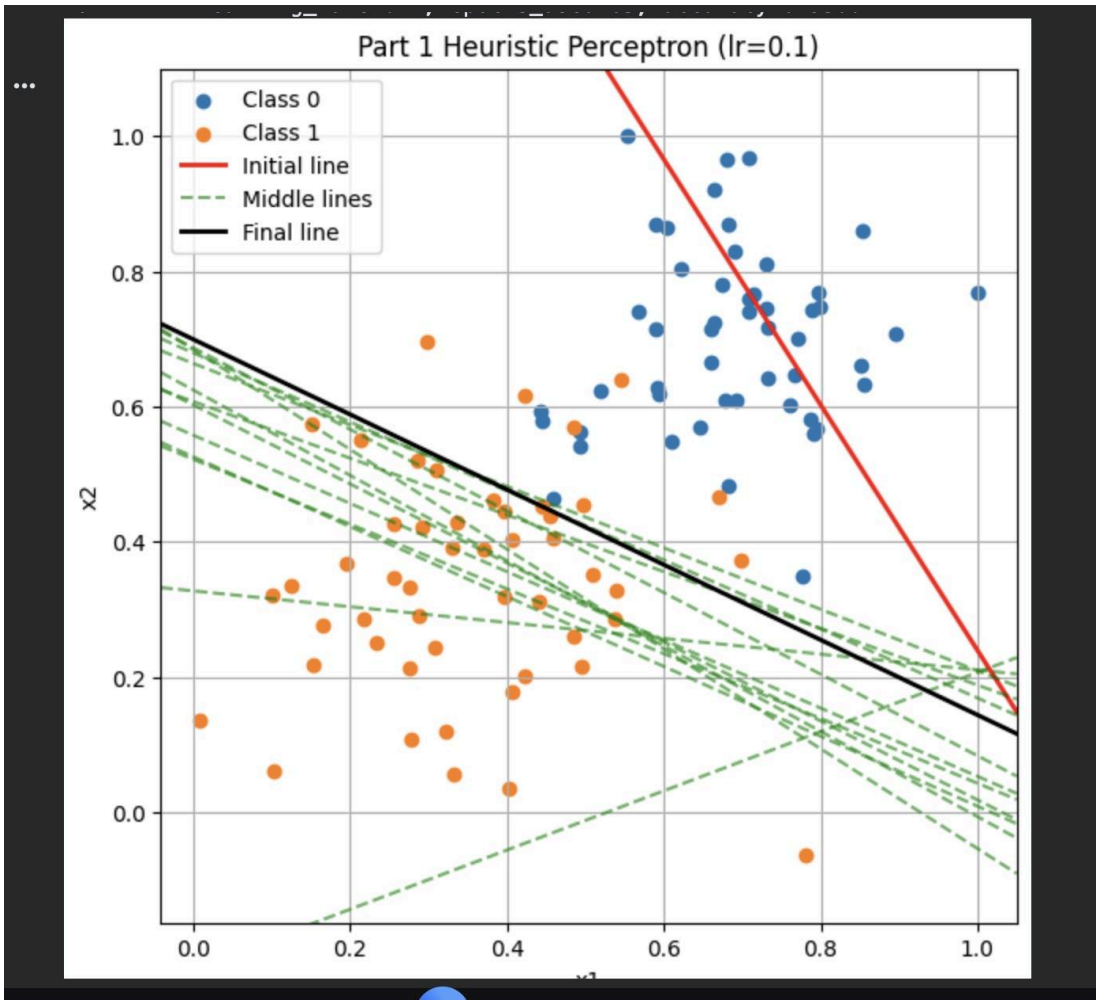
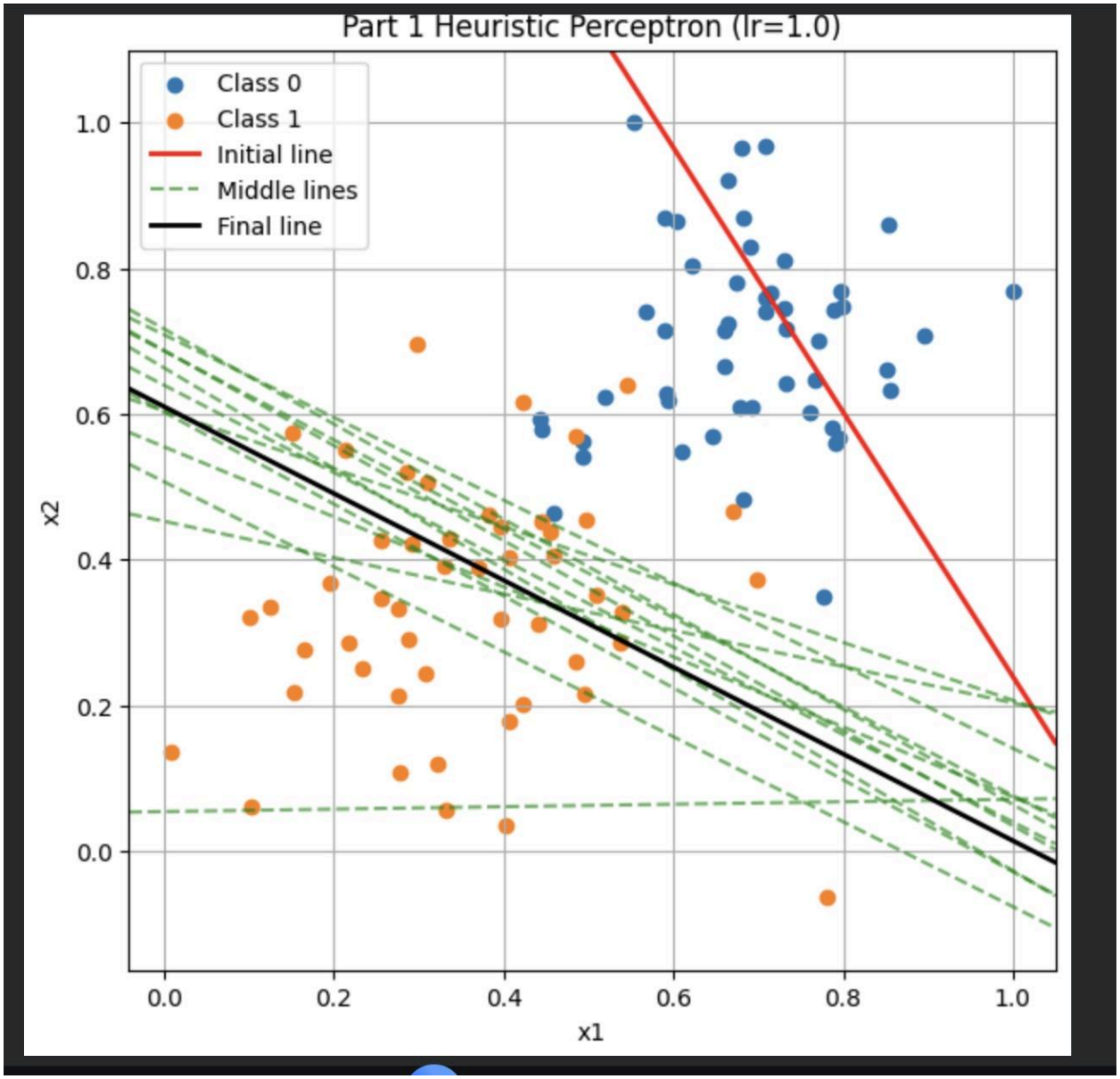



```
for lr in [0.01, 0.1, 1.0]:
    w, b, lines, epochs_used, accuracy = perceptron_part1(X, y, learning_rate=lr, max_epochs=65, seed=42)

    print(f'Part 1 -> learning_rate={lr}, epochs_used={epochs_used}, accuracy={accuracy:.4f}')

    plot_boundary_lines(X, y, lines, f'Part 1 Heuristic Perceptron (lr={lr})')
```


Part 1 Heuristic Perceptron (lr=0.01)
Class 0
Class 1
Initial line
Middle lines
Final line
1.0
0.8
0.6
0.4
0.2
0.0
x2
0.0
0.2
0.4
0.6
0.8
1.0


Part 1 Heuristic Perceptron (lr=0.1)
Class 0
Class 1
Initial line
Middle lines
Final line
x2
1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0

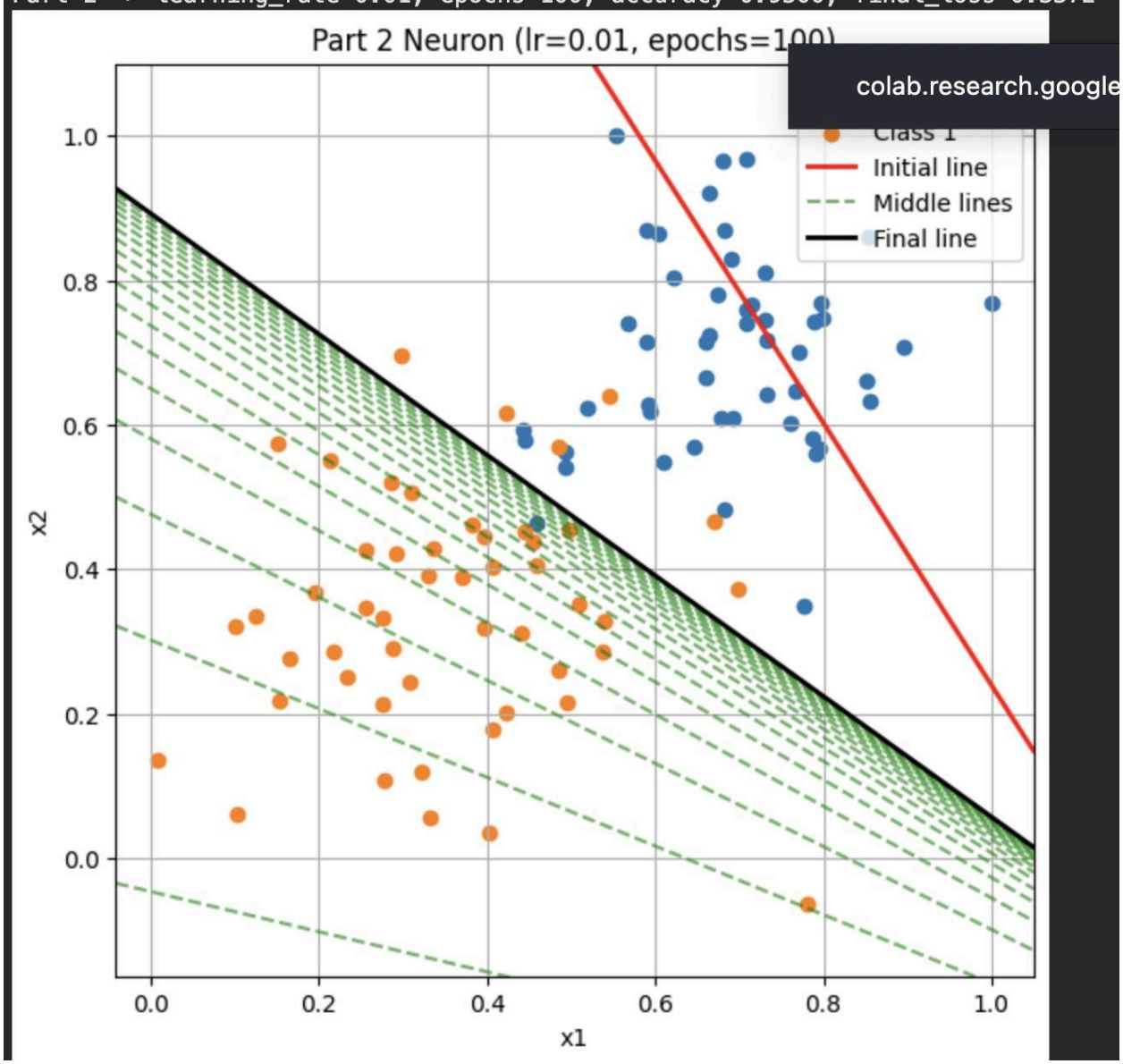

Part 1 Heuristic Perceptron (lr=1.0)
Class 0
Class 1
Initial line
Middle lines
Final line
1.0
0.8
0.6
0.4
0.2
0.0
x2
0.0
0.2
0.4
0.6
0.8
1.0
x1

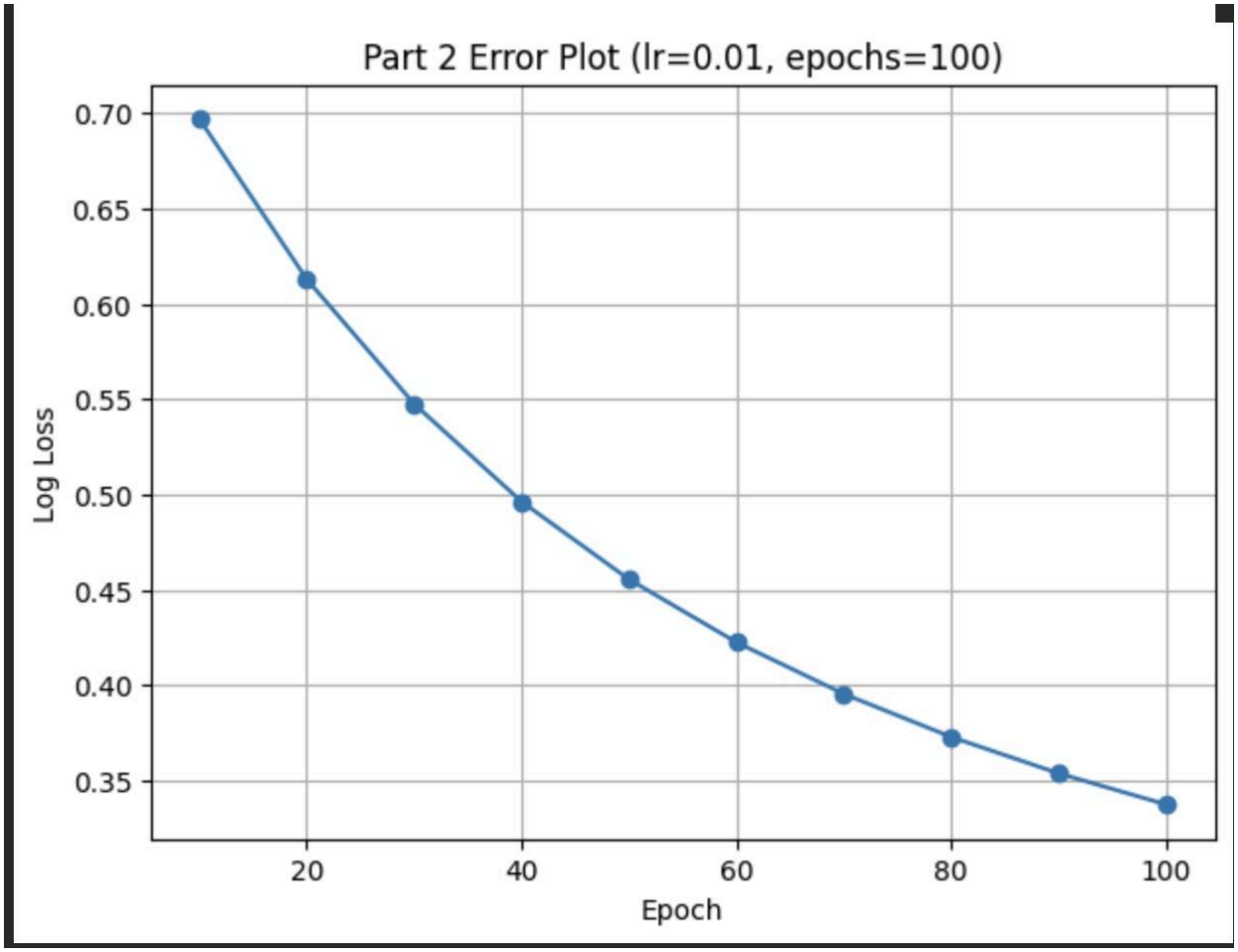
```python
def neuron_part2(X, y, learning_rate=0.1, max_epochs=100, seed=42):
    rng = np.random.default_rng(seed)

    while True:
        w = rng.uniform(-1, 1, size=X.shape[1])
        if abs(w[1]) > 0.2:
            break

    b = rng.uniform(-1, 1)
    lines = [(w.copy(), b)]
    epoch_points = []
    loss_points = []

    for epoch in range(1, max_epochs + 1):
        for xi, yi in zip(X, y):
            z = np.dot(w, xi) + b
            y_hat = sigmoid(z)
            error = yi - y_hat

            b = b + learning_rate * error
            w = w + learning_rate * error * xi

        lines.append((w.copy(), b))

        if epoch % 10 == 0:
            probs = sigmoid(X @ w + b)
            loss = log_loss(y, probs)
            epoch_points.append(epoch)
            loss_points.append(loss)

    probs = sigmoid(X @ w + b)
```

```python
settings = [
    (0.01, 100),
    (0.1, 100),
    (0.1, 200)
]

for lr, epochs in settings:
    w, b, lines, epoch_points, loss_points, accuracy = neuron_part2(
        X, y, learning_rate=lr, max_epochs=epochs, seed=42
    )

    print(f'Part 2 -> learning_rate={lr}, epochs={epochs}, accuracy={accuracy:.4f}, final_loss={loss_points[
    plot_boundary_lines(X, y, lines, f'Part 2 Neuron (lr={lr}, epochs={epochs})', show_every=5)
    plot_error_curve(epoch_points, loss_points, f'Part 2 Error Plot (lr={lr}, epochs={epochs})')
```

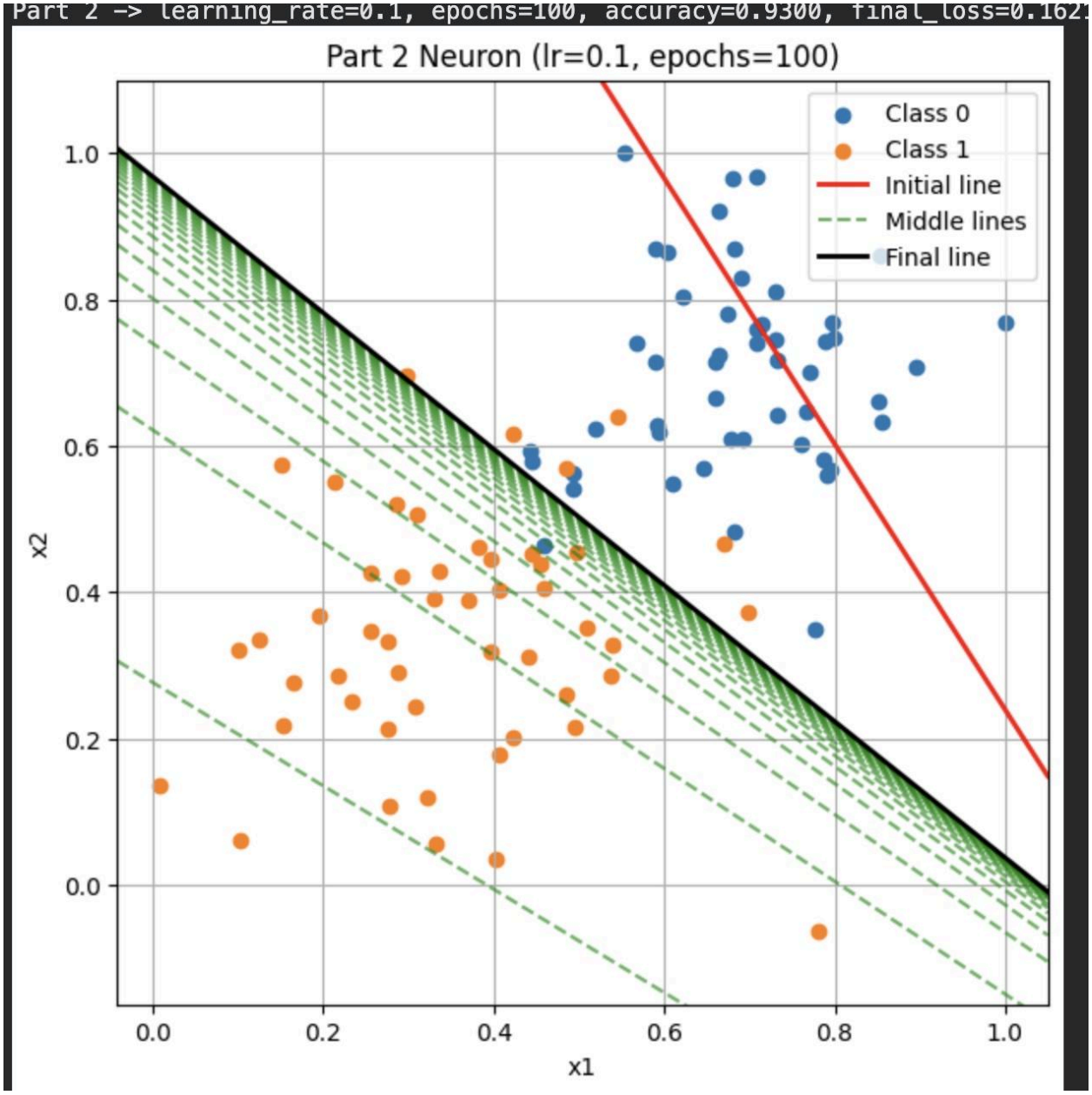
```
Part 2 -> learning_rate=0.01, epochs=100, accuracy=0.9300, final_loss=0.3372
```

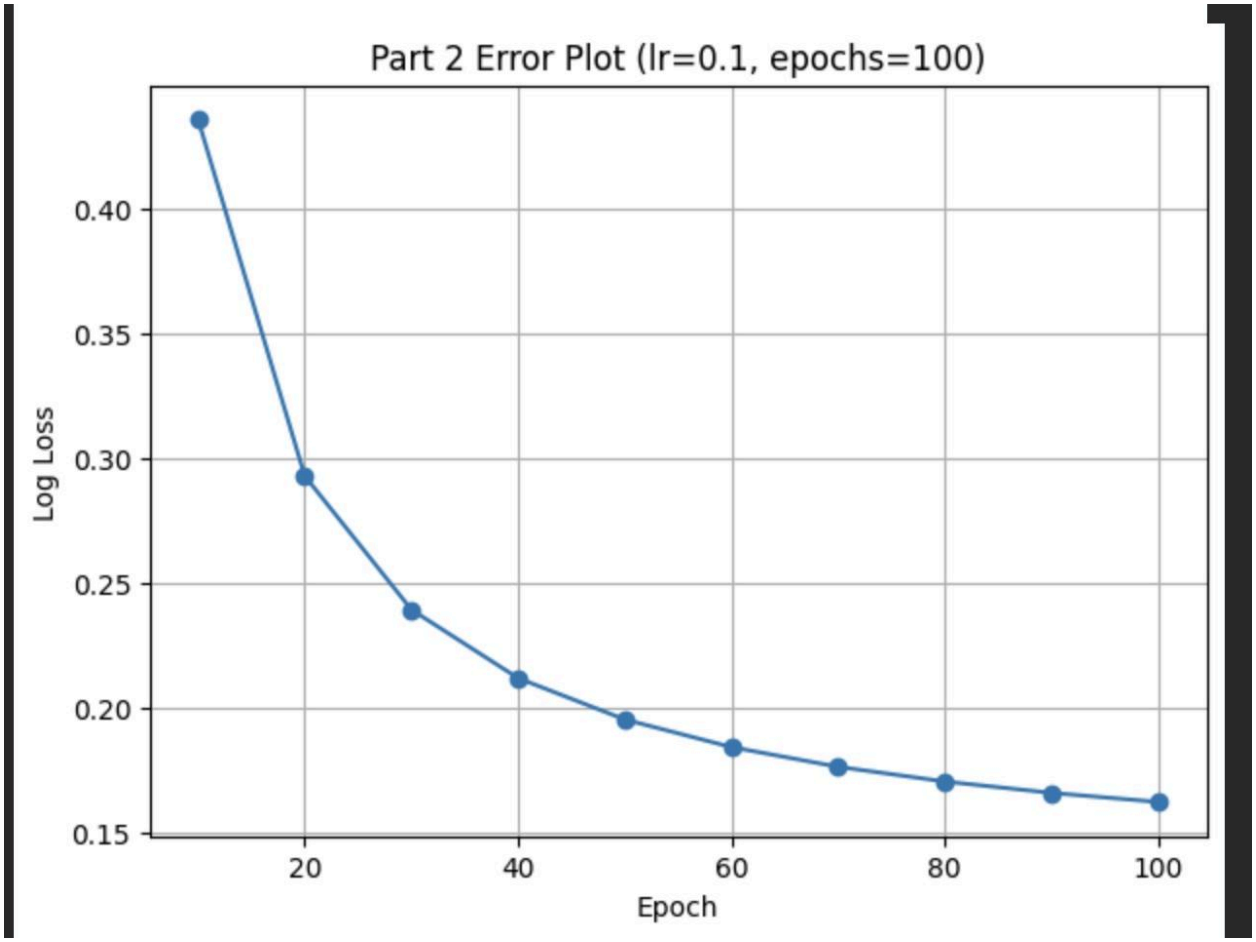
```
Part 2 -> learning_rate=0.01, epochs=100, accuracy=0.9300, final_loss=0.3372
```



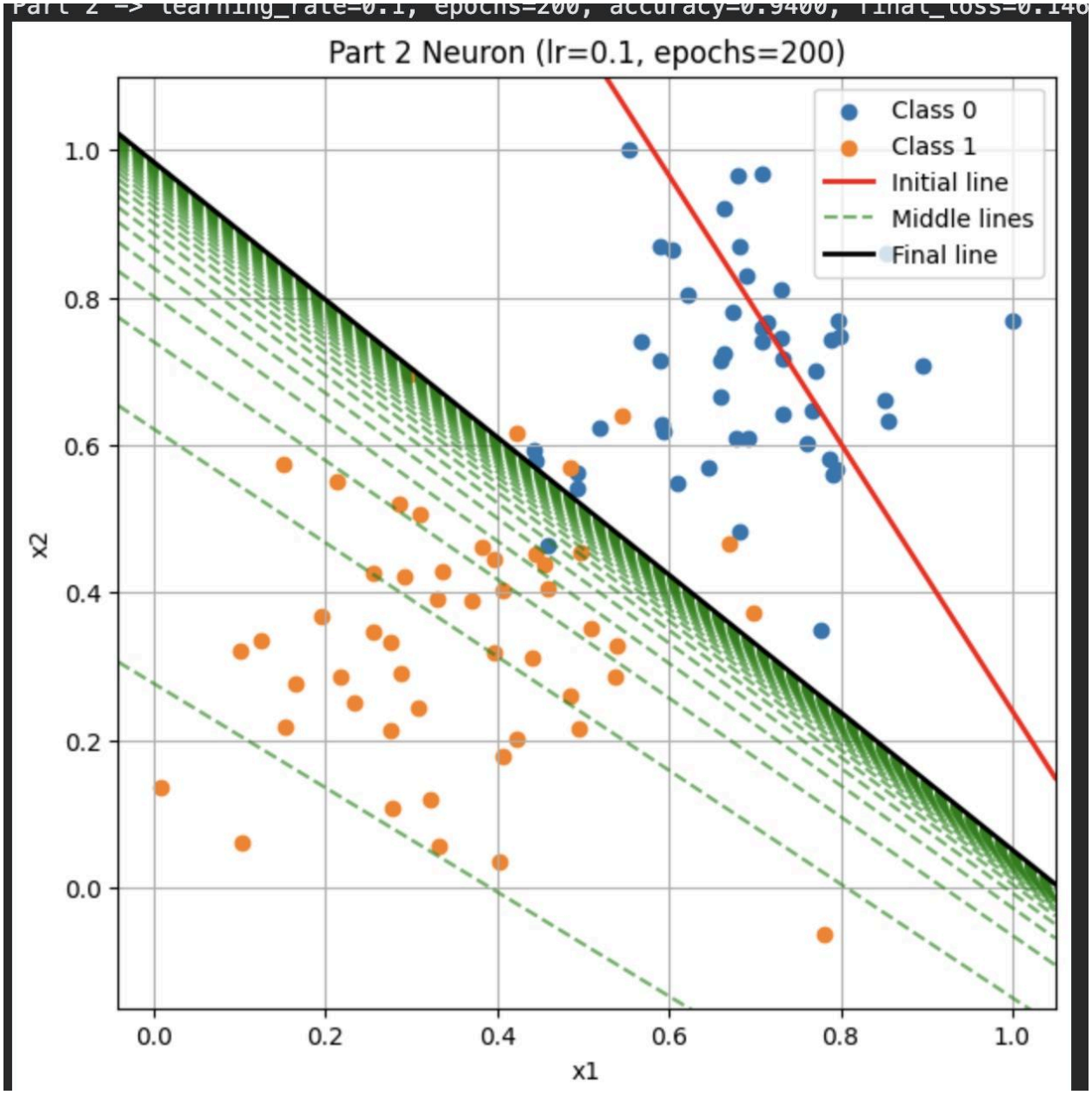

Part 2 Error Plot (lr=0.01, epochs=100)
Log Loss
0.70
0.65
0.60
0.55
0.50
0.45
0.40
0.35
20
40
60
80
100
Epoch

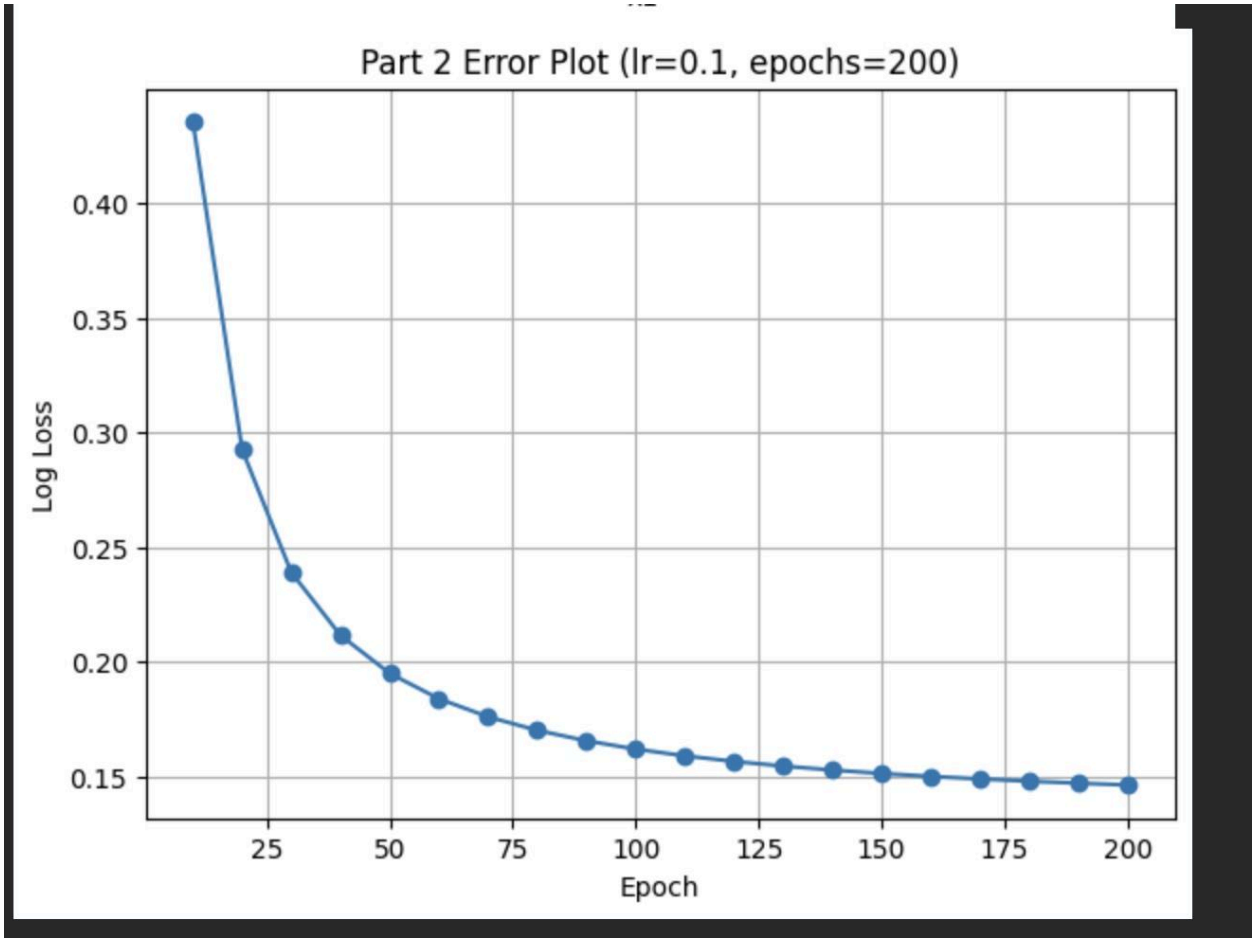
```
Part 2 -> learning_rate=0.1, epochs=100, accuracy=0.9300, final_loss=0.1621
```




Part 2 Error Plot (lr=0.1, epochs=100)
Log Loss
0.40
0.35
0.30
0.25
0.20
0.15
20
40
60
80
100
Epoch

```
Part 2 -> learning_rate=0.1, epochs=200, accuracy=0.9400, final_loss=0.146
```





For Part 1, I implemented the heuristic perceptron update rule. I tested learning rates of 0.01, 0.1, and 1.0. For each run, I plotted the initial line in red, the middle lines in dashed green, and the final line in black. The final boundary in each case moved toward a line that separated most of the two classes. A smaller learning rate changed the boundary more slowly, while a larger learning rate changed it faster. Overall, the plots showed how the perceptron gradually adjusted its boundary during training.

For Part 2, I used a neuron with sigmoid output. I tested different settings, including 0.01 with 100 epochs, 0.1 with 100 epochs, and 0.1 with 200 epochs. I also computed the log loss every 10 epochs and plotted the error curve. The error plots went down over time, which showed that the model was learning. The run with learning rate 0.1 performed better than 0.01, and increasing the epochs from 100 to 200 slightly improved the final result. This part showed that the neuron approach gave smoother learning and a clear decrease in loss over time.

Overall, both parts showed how the decision boundary changes as the weights are updated. Part 1 used a basic perceptron rule, while Part 2 used a sigmoid-based neuron and log loss. The experiments showed that learning rate affects how quickly the model changes, and the number of epochs affects how much the model improves. The results were reasonable, and the plots helped show the learning process clearly.